Universidad Mariano Gálvez

Alumno

Cristian Alexander Castro Morales

No. De Carné

0903-24-7112

Catedrático:

Ing. Bilder Fuentes

Carrera:

Facultad de Ingeniería en Sistemas 5to semestre

Sección:

“A”

Año:

2,026

GITHUB: https://github.com/kstroman/aplicaci-n-de-escritorio-que-gestione-al-menos-tres-datos.git

IDX

INDICE DE CONTENIDO

Seccion	Contenido	Pag.
1	Analisis y planteamiento del problema	3
2	Interfaz grafica de la aplicacion	5
3	Desarrollo de la aplicacion funcional	6
4	Capturas de pantalla en funcionamiento	9
5	Codigo fuente	11
6	Explicacion del codigo principal	15
7	Enlace al repositorio de GitHub	17

[1]ANALISIS Y PLANTEAMIENTO DEL PROBLEMA

Ejercicio 1 — Gestor de Productos con lista enlazada desde cero

1.1 Descripcion del problema

El Ejercicio 1 consiste en desarrollar una aplicacion de escritorio Windows Forms en C# (namespace ejercicio1elecciondelestudiante) que permita gestionar un catalogo de productos mediante una lista enlazada construida completamente desde cero, sin utilizar colecciones predefinidas de .NET. Cada registro almacena tres campos: Nombre (string), Precio (decimal) y Stock (int).

La aplicacion implementa las cinco operaciones fundamentales de la estructura: insercion al inicio, al final y en posicion intermedia, eliminacion y busqueda, con toda la logica reflejada en tiempo real sobre un DataGridView.

1.2 Estructura de la Lista Enlazada implementada

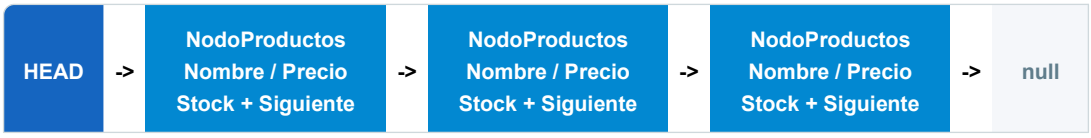

Figura 1 — Representacion de la lista enlazada con nodos NodoProductos

1.3 Operaciones implementadas y su complejidad

Operacion	Metodo en ListaProductos	Complejidad	Descripcion
Insertar al inicio	InsertarAlInicio(nombre, precio, stock)	O(1)	Nuevo nodo apunta a cabeza y pasa a s
Insertar al final	InsertarAlFinal(nombre, precio, stock)	O(n)	Recorre hasta Siguiente==null y enlaza
Insertar en posicion	InsertarEnPosicion(nombre,precio,stock,pos)	O(n)	Avanza pos-1 pasos y reconecta puntero
Eliminar	Eliminar(nombre) : bool	O(n)	Busca el nodo y salta su referencia en la
Buscar	Buscar(nombre) : NodoProductos	O(n)	Recorrido lineal insensible a mayusculas

1.4 Campos del nodo NodoProductos

Propiedad C#	Tipo	Descripcion	Ejemplo
Nombre	string	Nombre del producto	Monitor Samsung 27"
Precio	decimal	Precio unitario del producto	350.00
Stock	int	Unidades disponibles	12
Siguiente	NodoProductos	Referencia al siguiente nodo	(referencia interna)

Archivo .cs	Clase	Descripcion

1.5 Arquitectura del proyecto

NodoProductos.cs	class NodoProductos	Nodo: Nombre, Precio, Stock y referencia Siguiente
ListaProductos.cs	class ListaProductos	Lista enlazada con las 5 operaciones implementadas
Form1.cs	Form1 : Form	Formulario principal — DataGridView, TextBoxes, botones
Program.cs	Program (auto)	Punto de entrada: Application.Run(new Form1())

2.2 Inventario de controles

Control	Name (propiedad)	Configuracion	Funcion en la app
TextBox	txtNombre	PlaceholderText=Nombre	Entrada del nombre del producto
TextBox	txtPrecio	PlaceholderText=0.00	Entrada del precio (decimal)
TextBox	txtStock	PlaceholderText=0	Entrada del stock (entero)
NumericUpDown	nudPosicion	Minimum=0, Value=1	Posicion para InsertarEnPosicion()
Button	btnInicio	Text='Insertar al Inicio'	Llama lista.InsertarAlInicio()
Button	btnFinal	Text='Insertar al Final'	Llama lista.InsertarAlFinal()
Button	btnMedio	Text='Insertar en Posicion'	Llama lista.InsertarEnPosicion()
Button	btnEliminar	Text='Eliminar', BackColor=rojo	Llama lista.Eliminar()
Button	btnBuscar	Text='Buscar', BackColor=ver	eLlama lista.Buscar()
DataGridView	dgvLista	AutoSizeColumnsMode=Fill, R	eadOnly=trueMuestra Nombre, Precio, Stock
Label	lblResultado	ForeColor dinamico (verde/roj	) Mensajes de resultado

3 DESARROLLO DE LA APLICACION FUNCIONAL

Implementacion detallada de NodoProductos, ListaProductos y Form1

3.1 NodoProductos — estructura del nodo

La clase NodoProductos es la unidad atomica de la lista enlazada. Define los tres campos de datos del registro y la referencia Siguiente que es el puntero que enlaza un nodo con el proximo en la cadena:

NodoProductos.cs — namespace ejercicio1elecciondelestudiante

```
public class NodoProductos
{
public string Nombre { get; set; }
public decimal Precio { get; set; }
public int Stock { get; set; }
public NodoProductos Siguiente { get; set; } // puntero al siguiente
public NodoProductos(string nombre, decimal precio, int stock)
{
Nombre = nombre;
Precio = precio;
Stock = stock;
Siguiente = null; // por defecto no apunta a nadie
}
}
```

3.2 ListaProductos — constructor y campo cabeza

La clase ListaProductos mantiene un campo privado 'cabeza' que apunta al primer nodo (HEAD). Al inicializar la lista, cabeza es null lo que indica lista vacia:

ListaProductos.cs — esqueleto

```
public class ListaProductos
{
private NodoProductos cabeza; // HEAD: primer nodo de la lista
public ListaProductos()
{
cabeza = null; // lista vacia al inicio
}
// ... metodos a continuacion
}
```

3.3 InsertarAlInicio — O(1)

Crea un nuevo nodo y lo coloca al frente de la lista. El nuevo nodo apunta a quien era la cabeza, y luego cabeza pasa a ser el nuevo nodo. No requiere recorrer la lista, por eso es O(1):

```
public void InsertarAlInicio(string nombre, decimal precio, int stock)
{
NodoProductos nuevo = new NodoProductos(nombre, precio, stock);
nuevo.Siguiente = cabeza; // nuevo apunta al antiguo HEAD
cabeza = nuevo; // HEAD ahora es el nuevo nodo
}
```

3.4 InsertarAlFinal — O(n)

Si la lista esta vacia, el nuevo nodo se convierte directamente en cabeza. Si no, recorre toda la lista hasta el ultimo nodo (Siguiente == null) y le asigna el nuevo nodo como su sucesor:

```
public void InsertarAlFinal(string nombre, decimal precio, int stock)
{
NodoProductos nuevo = new NodoProductos(nombre, precio, stock);
if (cabeza == null) { cabeza = nuevo; return; }
NodoProductos actual = cabeza;
while (actual.Siguiente != null) // avanza hasta el ultimo nodo
actual = actual.Siguiente;
actual.Siguiente = nuevo; // enlaza el nuevo al final
}
```

3.5 InsertarEnPosicion — O(n)

Avanza (pos-1) pasos desde la cabeza. Si la posicion es 0 o menor, inserta al inicio. Si se pasa del final, inserta al final. En caso contrario, reconecta los punteros Siguiente para intercalar el nuevo nodo:

```
public void InsertarEnPosicion(string nombre, decimal precio, int stock, int pos)
{
if (pos <= 0) { InsertarAlInicio(nombre, precio, stock); return; }
NodoProductos nuevo = new NodoProductos(nombre, precio, stock);
NodoProductos actual = cabeza;
int indice = 0;
while (actual != null && indice < pos - 1)
{
actual = actual.Siguiente;
indice++;
}
if (actual == null) { InsertarAlFinal(nombre, precio, stock); return; }
nuevo.Siguiente = actual.Siguiente; // nuevo apunta al sucesor
actual.Siguiente = nuevo; // anterior apunta al nuevo
}
```

3.6 Eliminar — O(n)

Maneja el caso especial de eliminar la cabeza. Para cualquier otro nodo, mantiene dos punteros: anterior y actual. Cuando encuentra el nombre, hace que anterior.Siguiente salte a actual.Siguiente:

```
public bool Eliminar(string nombre)
{
if (cabeza == null) return false;
if (cabeza.Nombre == nombre) // caso especial: eliminar la cabeza
{
cabeza = cabeza.Siguiente;
return true;
}
NodoProductos anterior = cabeza;
NodoProductos actual = cabeza.Siguiente;
while (actual != null)
{
if (actual.Nombre == nombre)
{
anterior.Siguiente = actual.Siguiente; // salta el nodo eliminado
return true;
}
anterior = actual;
actual = actual.Siguiente;
}
return false; // nombre no encontrado
}
```

3.7 Buscar — O(n)

Recorre la lista comparando el Nombre de cada nodo con ToLower() para que la busqueda sea insensible a mayusculas. Retorna el nodo si lo encuentra o null:

```
public NodoProductos Buscar(string nombre)
{
NodoProductos actual = cabeza;
while (actual != null)
{
if (actual.Nombre.ToLower() == nombre.ToLower())
return actual; // encontrado
actual = actual.Siguiente;
}
return null; // no existe
}
```

— Prueba 6: Buscar() retorna null y lblResultado muestra texto en rojo.

5

CODIGO FUENTE

Codigo real del proyecto ejercicio1elecciondelestudiante

CODIGO REAL DEL ESTUDIANTE

El codigo presentado en esta seccion es el codigo fuente real escrito por el estudiante y utilizado en la aplicacion funcional.

NodoProductos.cs

NodoProductos.cs

```
namespace ejercicio1elecciondelestudiante
{
public class NodoProductos
{
public string Nombre { get; set; }
public decimal Precio { get; set; }
public int Stock { get; set; }
public NodoProductos Siguiente { get; set; }
public NodoProductos(string nombre, decimal precio, int stock)
{
Nombre = nombre;
Precio = precio;
Stock = stock;
Siguiente = null;
}
}
}
```

ListaProductos.cs

ListaProductos.cs

```
using System.Collections.Generic;
namespace ejercicio1elecciondelestudiante
{
public class ListaProductos
{
private NodoProductos cabeza;
public ListaProductos()
{
cabeza = null;
}
```

```
public void InsertarAlInicio(string nombre, decimal precio, int stock) {
NodoProductos nuevo = new NodoProductos(nombre, precio, stock);
nuevo.Siguiente = cabeza; cabeza = nuevo; } public void InsertarAlFinal(string
nombre, decimal precio, int stock) { NodoProductos nuevo = new
NodoProductos(nombre, precio, stock); if (cabeza == null) { cabeza = nuevo;
return; } NodoProductos actual = cabeza; while (actual.Siguiente != null)
actual = actual.Siguiente; actual.Siguiente = nuevo; } public void
InsertarEnPosicion(string nombre, decimal precio, int stock, int pos) { if
(pos <= 0) { InsertarAlInicio(nombre, precio, stock); return; } NodoProductos
nuevo = new NodoProductos(nombre, precio, stock); NodoProductos actual =
cabeza; int indice = 0; while (actual != null && indice < pos - 1) { actual =
actual.Siguiente; indice++; } if (actual == null) { InsertarAlFinal(nombre,
precio, stock); return; } nuevo.Siguiente = actual.Siguiente; actual.Siguiente
= nuevo; } public bool Eliminar(string nombre) { if (cabeza == null) return
false; if (cabeza.Nombre == nombre) { cabeza = cabeza.Siguiente; return true;
}
NodoProductos anterior = cabeza;
NodoProductos actual =
cabeza.Siguiente; while (actual !=
null) { if (actual.Nombre == nombre)
{
```

```
anterior.Siguiente = actual.Siguiente; return true;
} anterior = actual; actual = actual.Siguiente; }
return false; } public NodoProductos Buscar(string
nombre) { NodoProductos actual = cabeza; while
(actual != null) { if (actual.Nombre.ToLower() ==
nombre.ToLower()) return actual; actual =
actual.Siguiente; } return null; } public
List<NodoProductos> ObtenerTodos() {
List<NodoProductos> lista = new
List<NodoProductos>(); NodoProductos actual =
cabeza; while (actual != null) { lista.Add(actual);
actual = actual.Siguiente; } return lista; }
}
}
```

Form1.cs

Form1.cs

```
using System;
using System.Drawing;
using System.Windows.Forms;
namespace ejercicio1elecciondelestudiante
{
public partial class Form1 : Form
{
private ListaProductos lista = new ListaProductos();
public Form1()
{
InitializeComponent();
ConfigurarGrilla();
}
private void Form1 Load(object sender, EventArgs e) { }
private void ConfigurarGrilla()
{
dgvLista.Columns.Add("Nombre", "Nombre");
dgvLista.Columns.Add("Precio", "Precio");
dgvLista.Columns.Add("Stock", "Stock");
dgvLista.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
}
private void ActualizarGrilla()
{
dgvLista.Rows.Clear();
foreach (var p in lista.ObtenerTodos())
dgvLista.Rows.Add(p.Nombre, p.Precio.ToString(), p.Stock);
}
private bool ValidarCampos(out string nombre, out decimal precio, out int stock)
{
nombre = txtNombre.Text.Trim();
precio = 0;
stock = 0;
if (string.IsNullOrEmpty(nombre))
{
Mensaje("Ingresa un nombre.", true);
return false;
}
if (!decimal.TryParse(txtPrecio.Text, out precio))
{
Mensaje("Precio invalido.", true);
```

```
return false; } if (!int.TryParse(txtStock.Text, out stock)) {
Mensaje("Stock invalido.", true); return false; } return true; }
private void Mensaje(string msg, bool error = false) {
lblResultado.Text = msg; lblResultado.ForeColor = error ? Color.Red :
Color.Green; } private void btnInicio_Click(object sender, EventArgs
e) { if (!ValidarCampos(out string n, out decimal p, out int s))
return; lista.InsertarAlInicio(n, p, s); ActualizarGrilla();
Mensaje("'" + n + "' insertado al inicio."); } private void
btnFinal_Click(object sender, EventArgs e) { if (!ValidarCampos(out
string n, out decimal p, out int s)) return; lista.InsertarAlFinal(n,
p, s); ActualizarGrilla(); Mensaje("'" + n + "' insertado al final.");
} private void btnMedio_Click(object sender, EventArgs e) { if
(!ValidarCampos(out string n, out decimal p, out int s)) return;
lista.InsertarEnPosicion(n, p, s, (int)nudPosicion.Value);
ActualizarGrilla(); Mensaje("'" + n + "' insertado en posicion " +
nudPosicion.Value + "."); } private void btnEliminar_Click(object
sender, EventArgs e) { string nombre = txtNombre.Text.Trim(); if
(lista.Eliminar(nombre)) {
ActualizarGrilla(); Mensaje("'" +
nombre + "' eliminado."); } else
```

```
{ Mensaje("'" + nombre + "' no encontrado.",
true);
} } private void btnBuscar_Click(object sender,
EventArgs e) { var resultado =
lista.Buscar(txtNombre.Text.Trim()); if (resultado !=
null) Mensaje("Encontrado: " + resultado.Nombre +
" | Precio: " + resultado.Precio + " |
Stock: " + resultado.Stock); else
Mensaje("Producto no encontrado.",
true);
}
}
}
```

6

EXPLICACION DEL CODIGO PRINCIPAL

Funcionamiento detallado de cada metodo de la lista enlazada

6.1 InsertarAlInicio()

Se instancia un nuevo NodoProductos con los datos recibidos. La propiedad Siguiente del nuevo nodo se asigna a cabeza (el nodo que era primero hasta ese momento), de modo que el nuevo queda enlazado con el resto de la lista. A continuacion cabeza se actualiza para que apunte al nuevo nodo. Como no se necesita recorrer ninguna posicion, la operacion es O(1) independientemente del tamano de la lista.

6.2 InsertarAlFinal()

Si cabeza es null (lista vacia), el nuevo nodo se asigna directamente como cabeza y el metodo termina. Si la lista tiene elementos, se recorre nodo a nodo siguiendo las referencias Siguiente hasta encontrar el nodo cuyo Siguiente sea null, que es el ultimo. Entonces se le asigna el nuevo nodo como Siguiente, enlazandolo al final. Costo O(n) porque el recorrido depende del numero de nodos.

6.3 InsertarEnPosicion()

Primero se verifica si pos es menor o igual a 0 y en ese caso delega en InsertarAlInicio. De lo contrario avanza con un contador desde cabeza hasta llegar al nodo en indice == pos-1 (el nodo justo antes del punto de insercion). Si actual llega a null antes de alcanzar la posicion, delega en InsertarAlFinal. En el caso normal, se asigna nuevo.Siguiente = actual.Siguiente (el nuevo apunta al que estaba en esa posicion) y luego actual.Siguiente = nuevo (el anterior apunta al nuevo), reconectando correctamente la cadena.

6.4 Eliminar()

El metodo trata primero el caso especial de que el nodo a eliminar sea cabeza: simplemente hace que cabeza apunte a su siguiente, dejando el nodo anterior sin referencia. Para cualquier otro nodo se usan dos punteros: anterior (el nodo previo) y actual (el candidato). Al encontrar el Nombre coincidente, se hace anterior.Siguiente = actual.Siguiente, que salta al nodo eliminado de la cadena. Sin ninguna referencia apuntando a el, el recolector de basura de .NET lo libera. Retorna true si se elimino, false si el nombre no existia.

6.5 Buscar()

Recorre la lista desde cabeza comparando actual.Nombre.ToLower() con nombre.ToLower(), lo que hace la busqueda insensible a mayusculas y minusculas. Al encontrar una coincidencia retorna el nodo directamente, permitiendo al formulario acceder a sus propiedades Nombre, Precio y Stock. Si se agota la lista sin encontrar nada, retorna null y el formulario muestra el mensaje 'Producto no encontrado.' con ForeColor rojo.

6.6 ActualizarGrilla() — sincronizacion con la UI

Este metodo privado del formulario es el puente entre la estructura de datos y lo que ve el usuario. Cada vez que la lista cambia (insercion, eliminacion), se llama ActualizarGrilla(), que primero limpia todas las filas del dgvLista

con Rows.Clear() y luego itera sobre lista.ObtenerTodos() agregando una fila por cada nodo con sus tres valores: Nombre, Precio.ToString() y Stock. Esto garantiza que la grilla siempre refleja fielmente el estado real de la lista enlazada.

6.7 ValidarCampos() — validacion de entradas

Antes de cualquier insercion se ejecuta ValidarCampos() para verificar tres condiciones: que txtNombre no este vacio (string.IsNullOrEmpty), que txtPrecio sea convertible a decimal (decimal.TryParse) y que txtStock sea convertible a entero (int.TryParse). Si cualquiera falla, se muestra el mensaje de error en rojo mediante Mensaje(msg, true) y el metodo retorna false, cancelando la operacion de insercion sin modificar la lista.

7

ENLACE AL REPOSITORIO DE GITHUB

Codigo fuente publicado y compartido con el profesor

https://github.com/[TuUsuario]/ejercicio1elecciondelestudiante

INSTRUCCION

Reemplaza [TuUsuario] con tu nombre real de GitHub antes de entregar. Verifica que el enlace abre en el navegador y que los tres archivos .cs son visibles: NodoProductos.cs, ListaProductos.cs y Form1.cs.

Pasos para subir a GitHub

Paso	Accion
1	Crear cuenta en github.com y verificar el correo electronico
2	Instalar Git para Windows desde git-scm.com (todas las opciones por defecto)
3	git config --global user.name 'Tu Nombre' && git config --global user.email 'tu@correo.com'
4	En GitHub: + > New repository > ejercicio1elecciondelestudiante > Public > Create
5	En la carpeta del proyecto: git init && git add . && git commit -m 'Primera version'
6	git remote add origin https://github.com/[TuUsuario]/ejercicio1elecciondelestudiante.git
7	git branch -M main && git push -u origin main
8	Verificar en el navegador que NodoProductos.cs, ListaProductos.cs y Form1.cs son visibles
9	Si es privado: Settings > Collaborators > Add people > agregar al profesor

Lista de verificacion final

Elemento	OK
NodoProductos.cs con campos Nombre, Precio, Stock y Siguiente	[ ]
ListaProductos con InsertarAlInicio implementado y funcional	[ ]
ListaProductos con InsertarAlFinal implementado y funcional	[ ]
ListaProductos con InsertarEnPosicion implementado y funcional	[ ]
ListaProductos con Eliminar implementado y funcional	[ ]
ListaProductos con Buscar implementado y funcional	[ ]
Form1 con ConfigurarGrilla() configurando las 3 columnas correctamente	[ ]

Form1 con ActualizarGrilla() sincronizando la lista con el DataGridView	[ ]
Form1 con ValidarCampos() validando nombre, precio y stock	[ ]
Prueba 1 — Insertar al inicio ejecutada y captura tomada	[ ]
Prueba 2 — Insertar al final ejecutada y captura tomada	[ ]
Prueba 3 — Insertar en posicion ejecutada y captura tomada	[ ]
Prueba 4 — Buscar producto existente ejecutada y captura tomada	[ ]
Prueba 5 — Eliminar ejecutada y captura tomada	[ ]
Prueba 6 — Buscar producto inexistente ejecutada y captura tomada	[ ]
Repositorio GitHub publico o con acceso al profesor	[ ]
Enlace de GitHub incluido y verificado en el informe	[ ]
Informe exportado a PDF y listo para subir a Canvas	[ ]